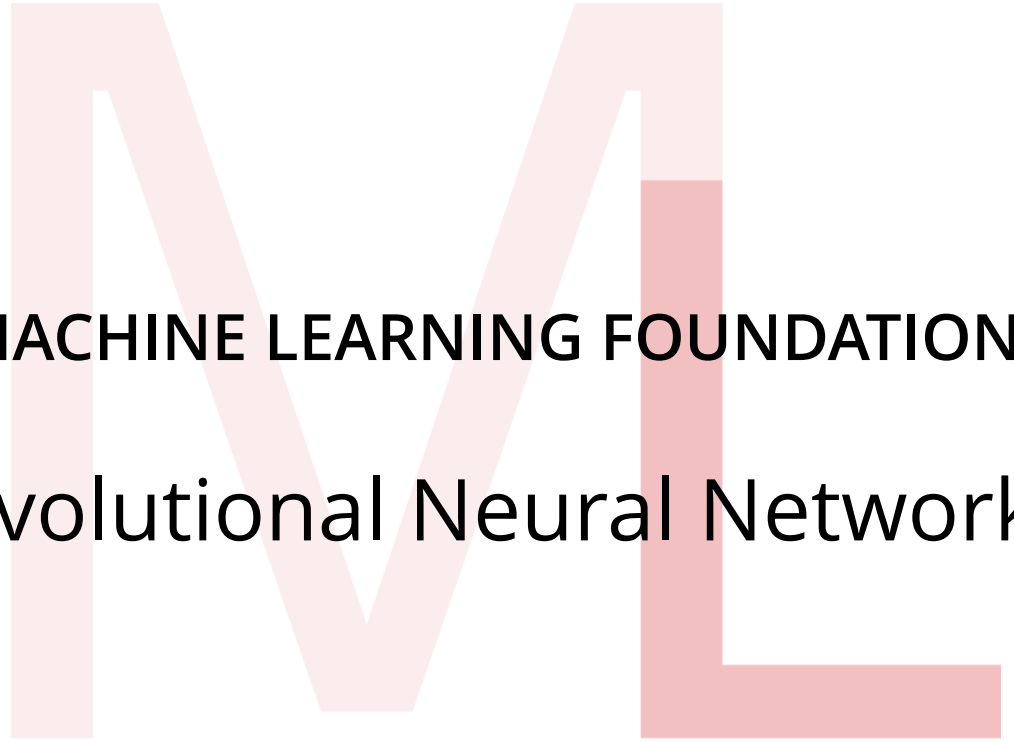


MACHINE LEARNING FOUNDATIONS

Convolutional Neural Networks (I)

University Master's Degree in
Machine Learning

upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Agenda

01 Recap

02 Image Convolution

03 Padding and stride

04 Multiple input/outputs

05 Convolutional layer

06 Pooling layer

Recap

- Usual Neural Networks (shallow or deep) allow us to deal with image classification problems
- Several issues
  - The image must be vectorized (flattened) to be processed
  - We can deal with RGB, but again each color must be concatenated one after another
  - The number of parameters grows with the size of the image
- NNs are not the most appropriate models to deal with images
  - Main reason: they do not exploit the spatial information of the images

Recap

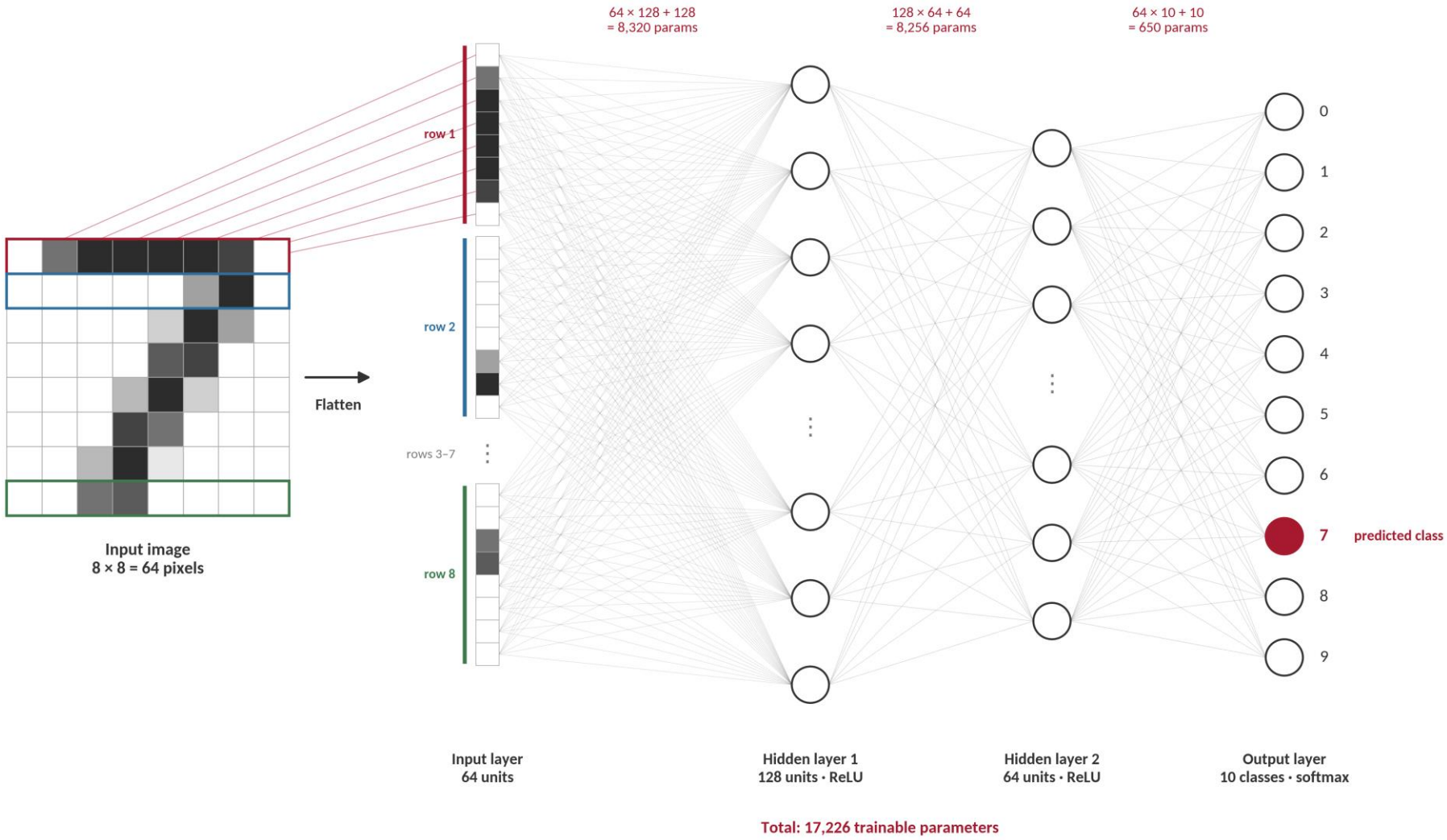

64 × 128 + 128
= 8,320 params
128 × 64 + 64
= 8,256 params
64 × 10 + 10
= 650 params
row 1
row 2
rows 3–7
row 8
Flatten
Input image
8 × 8 = 64 pixels
0
1
2
3
4
5
6
7
8
9
predicted class
Input layer
64 units
Hidden layer 1
128 units · ReLU
Hidden layer 2
64 units · ReLU
Output layer
10 classes · softmax
Total: 17,226 trainable parameters

Image convolution

- When using NNs with images, pixels are used directly as features
- This idea works for simple images (MNIST), but fails on more complex ones
- Idea: add a previous stage that extracts features from the image
  - Features are more informative than raw pixels
- Later, these features are flattened and fed to an MLP
- We change the concept of neuron to something that has these properties:
  - Translation equivariance: a shift in the image should produce a shift in the detection
  - Locality: we should not have to look very far away from the pixel where the pattern occurs in order to detect it
- Concept: image convolution

Image convolution

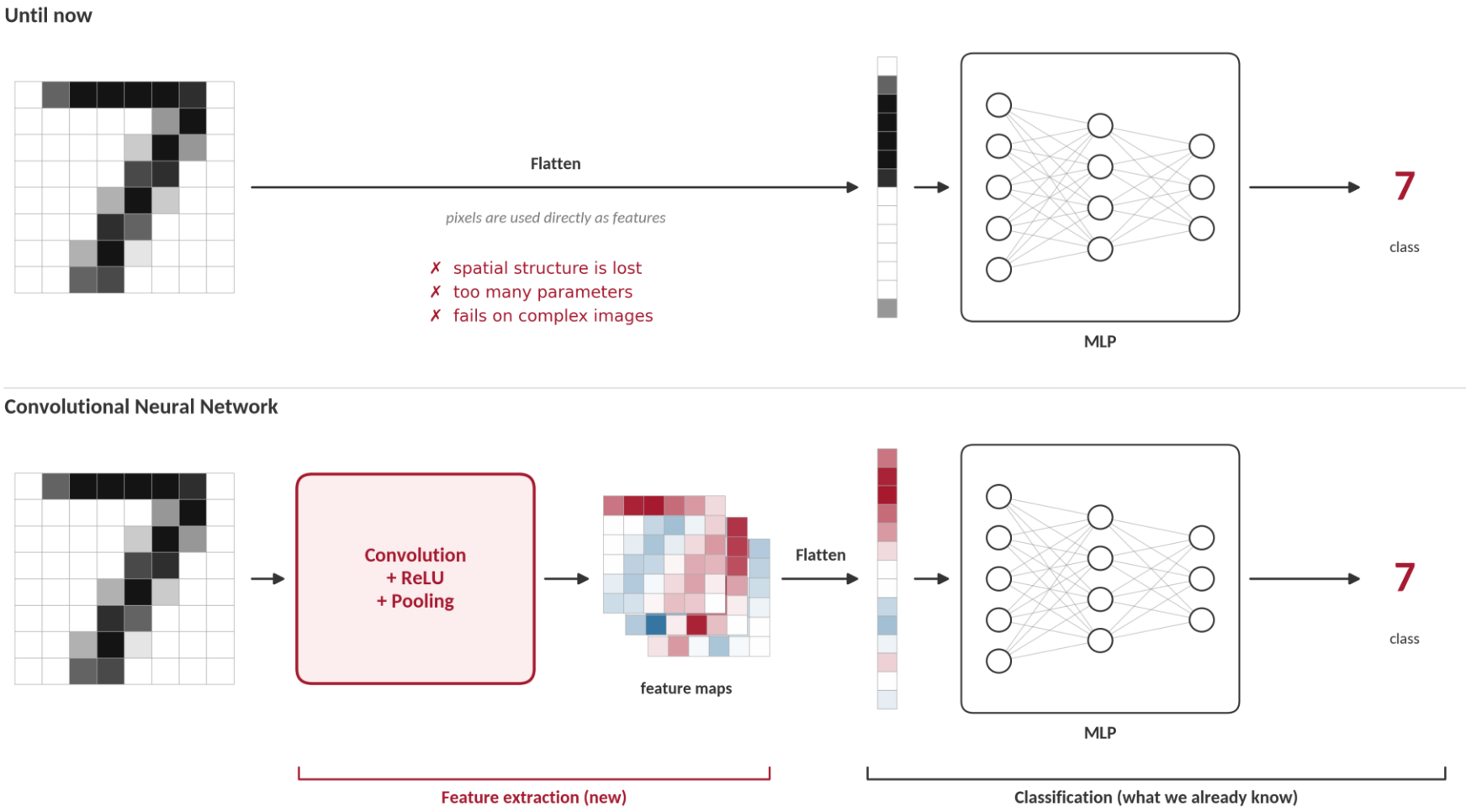
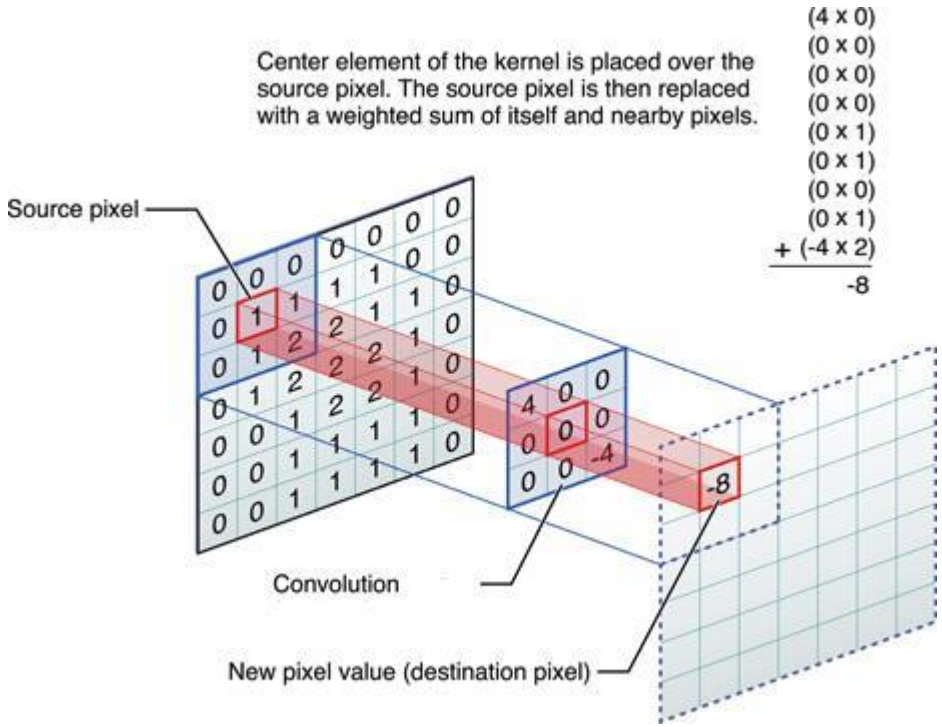


Image convolution

- In image processing and computer vision, convolution of images is one the most common technique
  - Noise removal
  - Blurring
  - Edge detection
  - Very small pattern detection
- For applying a convolution we need
  - Original image
  - Convolutional kernel (size and values)



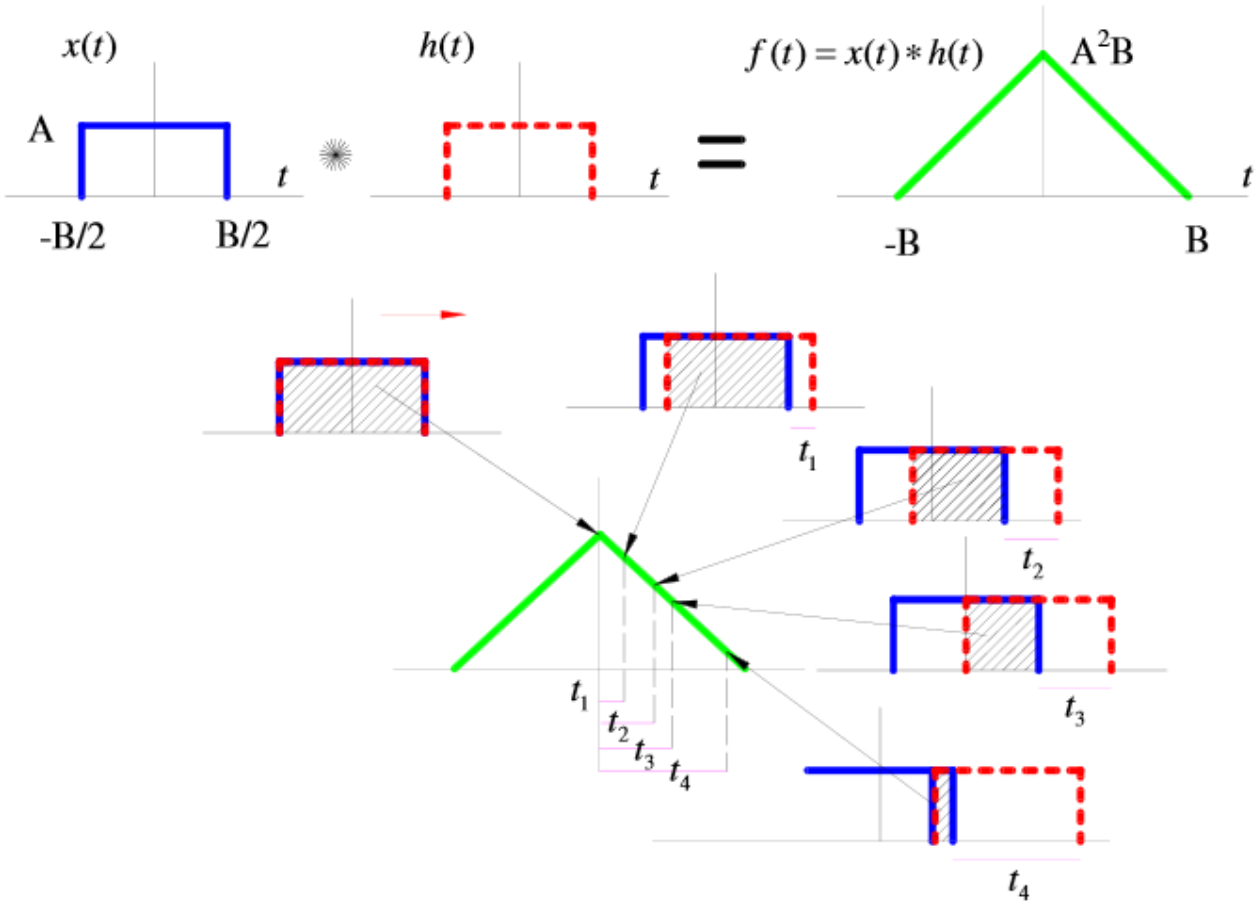
Mathematical definition

- The convolution between two functions f and g is defined as

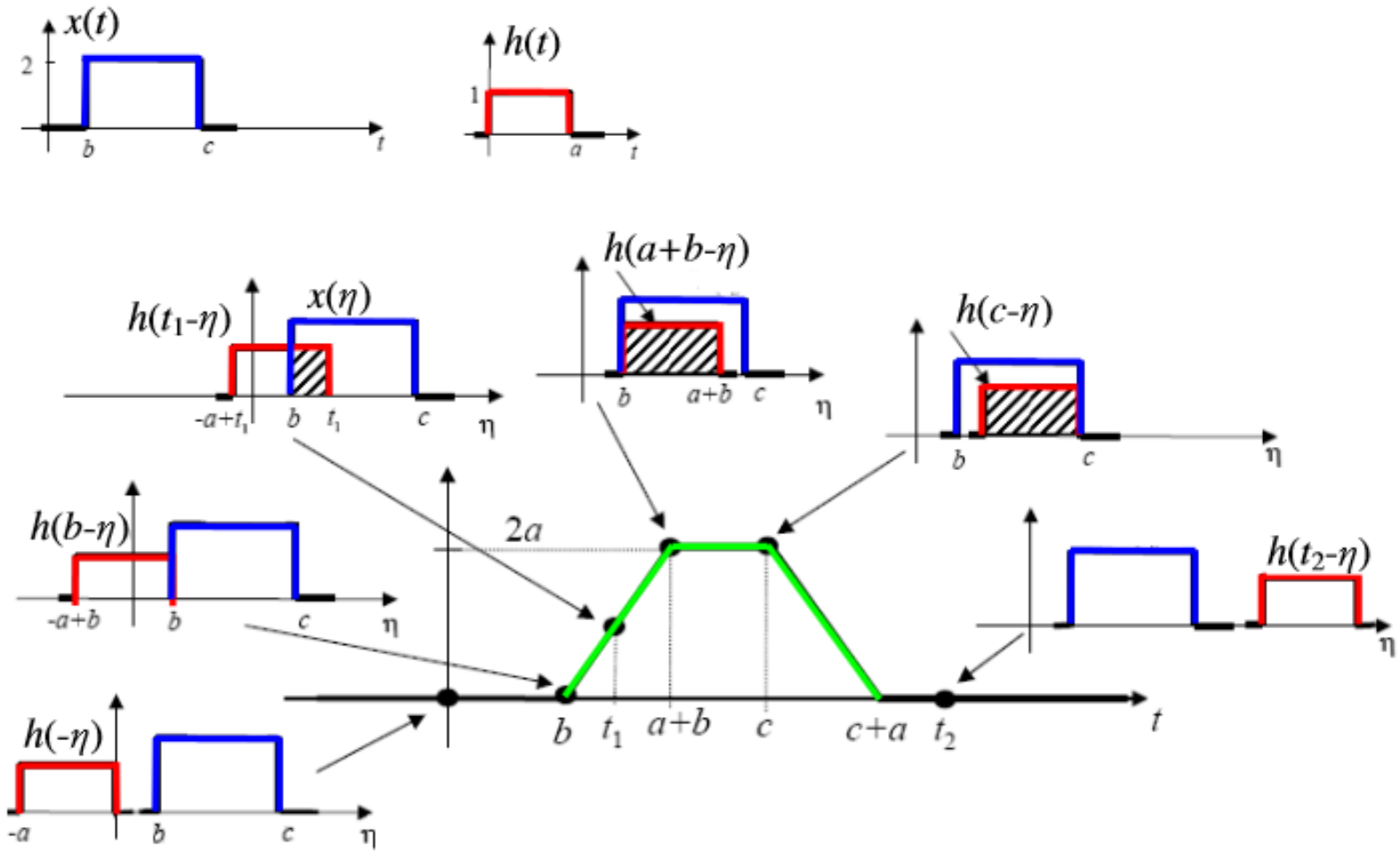
$$f * g(i) = \int_{-\infty}^{\infty} f(x)g(i-x)dx$$

- Main ideas
  - The function g is flipped along the vertical axis
  - The function g is shifted to i
  - Both functions are multiplied
  - The “sum” of the overlap is computed
- The convolution measures how “similar” (overlap) are both functions when g is shifted around i

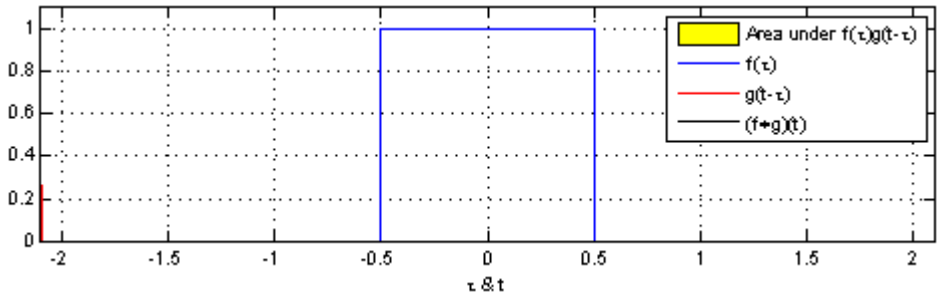
Mathematical definition

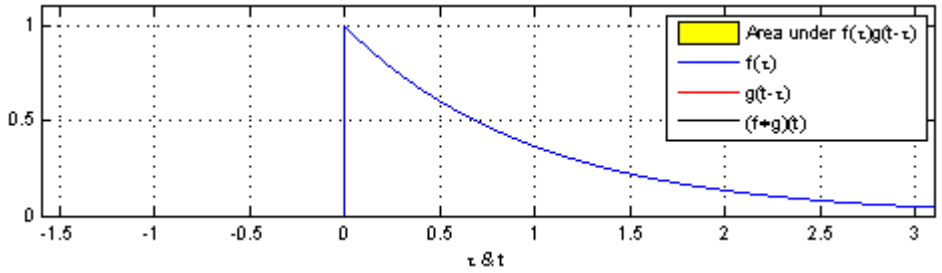


Mathematical definition



Mathematical definition





Mathematical definition

- In computer vision, images can be seen as 2D functions
- Definition of convolution for two-dimensional functions

$$f * g(i,j) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x,y)g(i-x,j-y)dxdy$$

Mathematical definition

- However, images are discrete functions, so we need to adapt the definition

$$f * g(i,j) = \sum_m \sum_n f(m,n) g(i-m, j-n)$$

- In practice
  - We are not interested in analyzing the whole domain of f and g, but somewhere local around each pixel location, so kernel functions tend to be very small
  - The flipped of g is not usually performed (we talk about correlation rather than convolution), so we calculate $g(i+m, j+n)$
  - The origin of coordinates in both f and g are different

Image convolution in practice

- Usually, what we have is the following
  - f: image that we want to convolve (analyze)
  - g: a very small image (kernel) of size 3x3, 5x5, 7x7, containing specific values for specific behaviour
  - Procedure: for each pixel (i,j) in f
    - The kernel is shifted and centered in (i,j)
    - The values of g are multiplied by the corresponding pixels in f
    - The result is located in a new image
- Example: for a 3x3 kernel g

$$f * g(i,j) = \sum_{m=-1}^{1} \sum_{n=-1}^{1} f(i+m, j+n) g(m,n)$$

Image convolution in practice

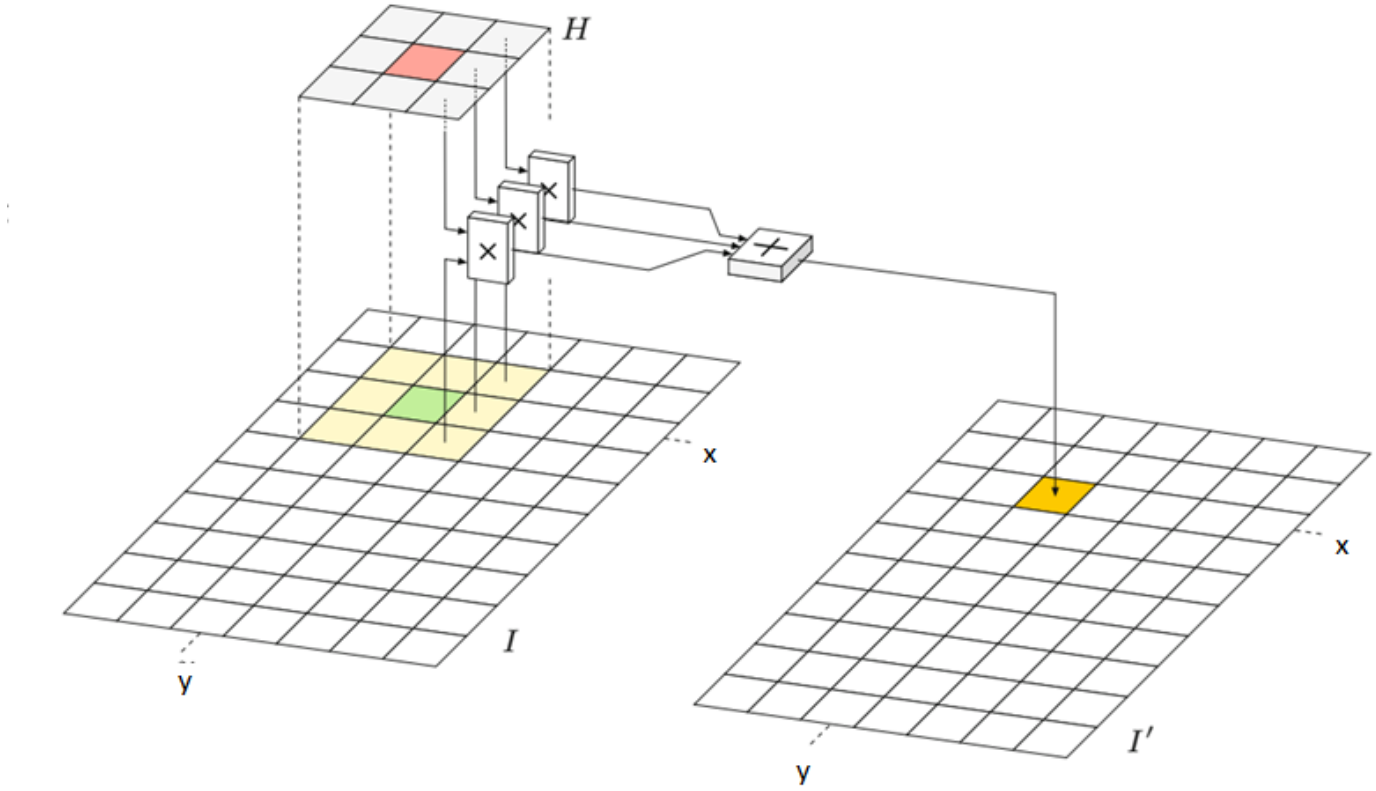


Image convolution in practice

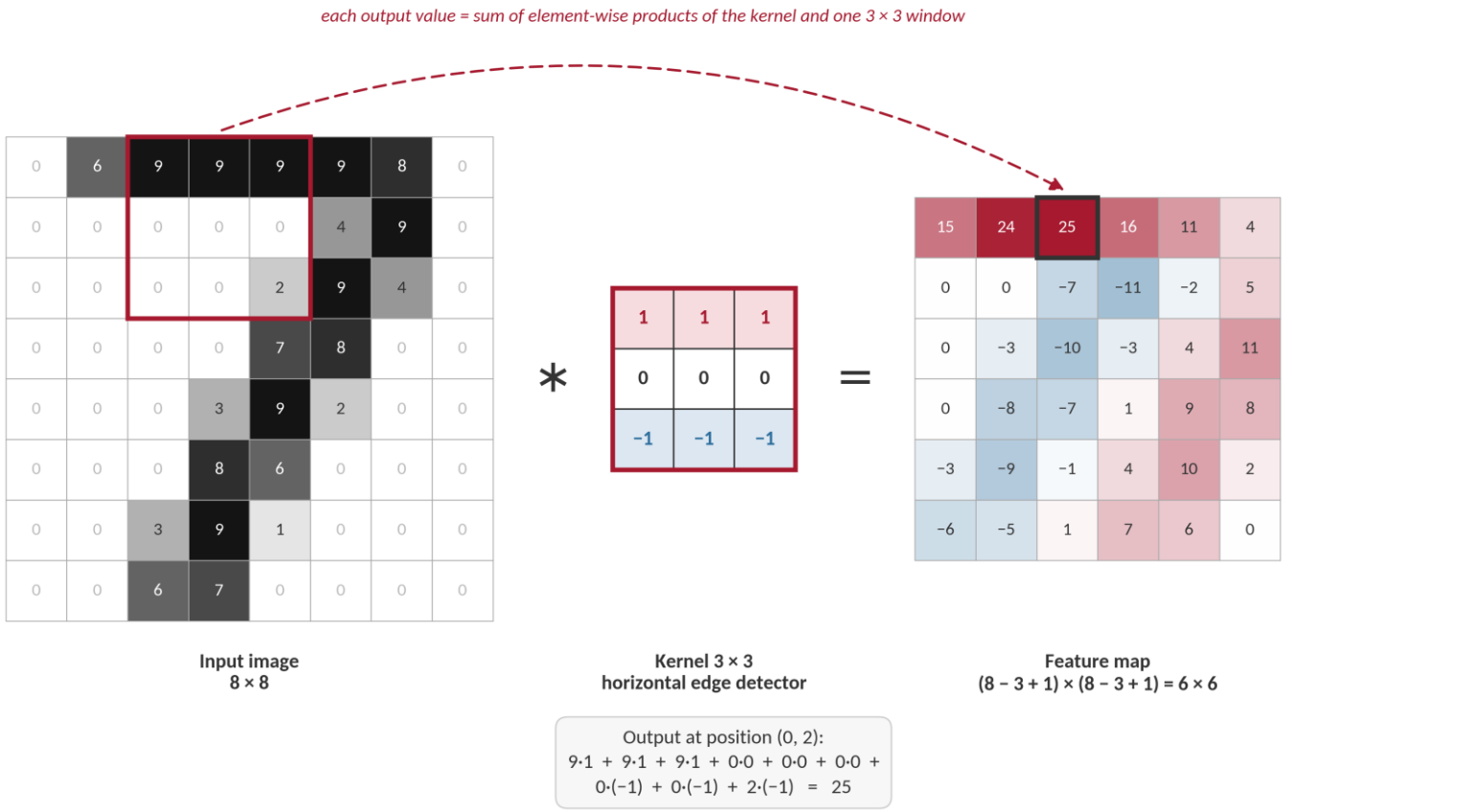


Image convolution in practice

- Example: given an imagen of 128x128 pixels, the convolution with a
  - 3x3 kernel generates an image of 126x126 pixels
  - 5x5 kernel generates an imagen of 124x124 pixels
- In general, if we han image of $n_h \times n_w$ pixels and a kernel of $k_h \times k_w$ elements, the convolution will be composed of

$$(n_h - k_h + 1) \times (n_w - k_w + 1)$$

Kernel examples

- As told before, image convolutions are very common in image processing
- Very well-known kernels
  - Kernel for blurring



Original



$1/9$

1	1	1
1	1	1
1	1	1



$1/25$

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

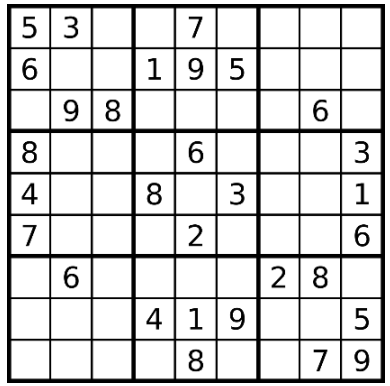


$1/49$

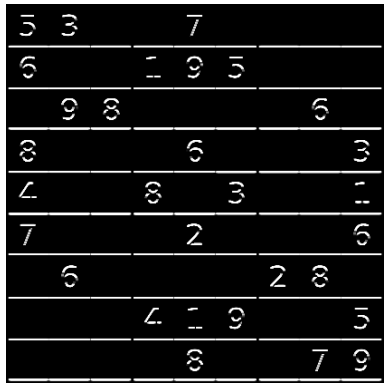
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1

Kernel examples

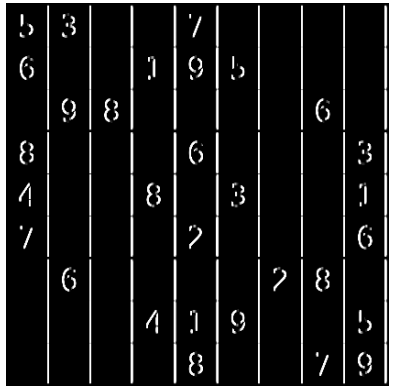
- Kernels for Edge detection



Original



1	1	1
0	0	0
-1	-1	-1



1	0	-1
1	0	-1
1	0	-1

Kernel examples

- Kernel for image sharpening



Original

0	-1	0
-1	5	-1
0	-1	0

Learning convolutions

- As we have seen, different kernels produce different convolutions (behaviours)
- We should understand image convolution as the smallest possible feature extractor
  - The processing is more complex than just considering the pixel itself
  - The patterns that a convolution can learn are those that can be detected by (linearly) operating a pixel and its neighbours.
- When we combine convolution with neural networks, we can think on the kernel as a set of parameters that can be learned using gradient descent
  - Gradients can be computed with backprop. as in MLP
- What kernel will the neural network learn?
  - That which is useful for the classification task

Padding and stride

- Recall that, when applying the convolution to an image, we obtain a new image that is almost the same size
  - We loose the pixels on the perimeter of the image depending on the size of the kernel
- Padding and stride are techniques that allow us to model the size of the convolution
  - Padding: allowing to obtain a new image being the same size as the original
  - Stride: allowing to obtain much smaller images by controlling the step

Padding

- The fact of loosing some pixels when applying a convolution is not very important, but sometimes it is not desirable
- Padding: straightforward solution that consists inadding extra pixels of filler around the boundary of our input image
- Example: if an original image 128x128, convolved by a 3x3 kernel produces an image of 126x126
  - We can add 1 extra pixel around the input image, so that the padded image will be 130x130
  - When convolving with the 3x3 kernel, the resulted image will be 128x128

Padding

- In general, if we add p_h rows of padding (half on top and on bottom) and p_w columns (half on left and on right), the output of the convolution will be

$$(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1)$$

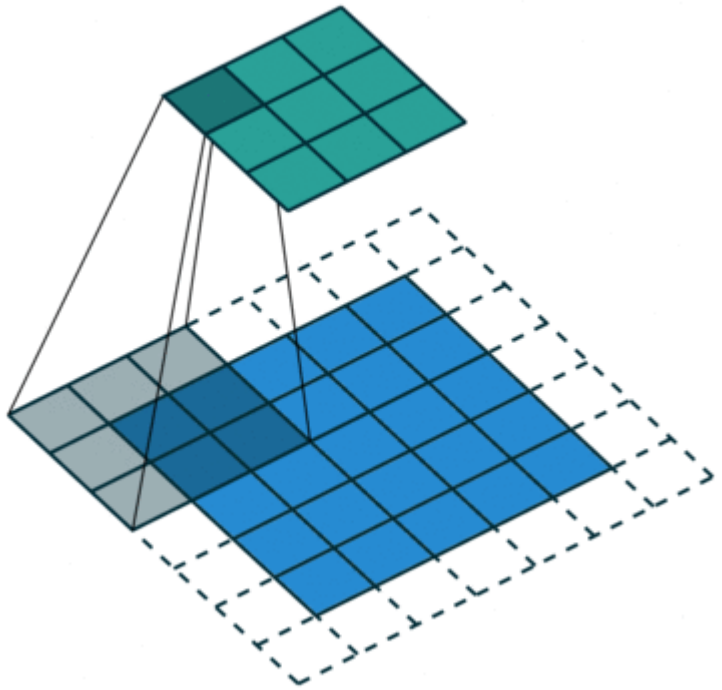
- Usually, we will have $p_h = k_h - 1$ and $p_w = k_w - 1$,

Stride

- When computing the convolution, we start at the upper-left corner and slide the kernel over all locations (both down and right), one at a time
- Sometimes, we want the kernel to be moved more tan one element at a time, skipping intermediate locations
  - For the sake of efficiency
  - Or to downsample the resulting image
- Stride refers to the number of rows and columns traversed per slide

Stride

- Example of a convolution with padding = 2 and stride = 2



Stride

- In general, if we have a stride for the height of s_h and a stride for the width of s_w, the output shape will be

$$\lfloor (n_h - k_h + p_h + s_h)/s_h \rfloor \times \lfloor (n_w - k_w + p_w + s_w)/s_w \rfloor$$

- In the case of $p_h = k_h - 1$ and $p_w = k_w - 1$, we have

$$\lfloor (n_h + s_h - 1)/s_h \rfloor \times \lfloor (n_w + s_w - 1)/s_w \rfloor$$

- In case n_h and n_w are divisible by the strides, we have

$$(n_h/s_h) \times (n_w/s_w)$$

Padding and stride

- As a summary
  - Padding is often used to give the output the same size as the input
  - We tipically pick $p_h = p_w$ and we use "a padding of p" ($p = \frac{p_h}{2} = \frac{p_w}{2}$)
  - We tipically pick $s_h = s_w$ and we use "a stride of s" to reduce the size of the image in 1/s (approximately)
  - By default, padding is 0 and stride is 1

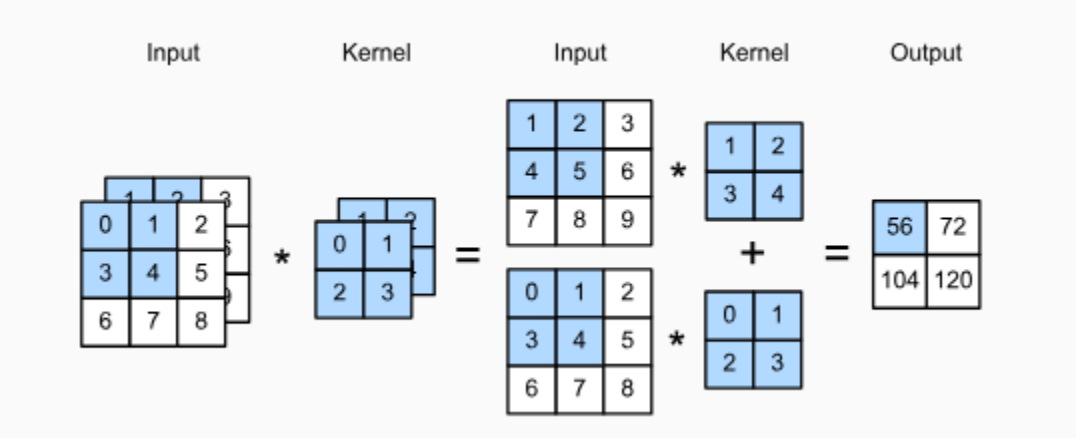
Multiple output/input

- So far, we have worked with image convolution with
  - One single input (the image in grayscale)
  - One single output (the convolved image by means of the kernel)
- Going from single output to multiple outputs is straightforward
  - If we want the output to be composed by c images, we should apply c convolutions with c different kernels

Multiple output/input

- To understand multiple input, one can think on RGB images
  - The size of an RGB image is 3xHxW
- How can we apply a convolution to a 3xHxW image?
  - First idea would be to apply the same kernel to each color component
- In DL, the size of the kernel is increased to fit the number of channels in the image
- For example, if we want to apply a 5x5 kernel to a 3xHxW image
  - The kernel size will be 3x5x5
  - The size of the convolution will be HxW (if padding)

Multiple output/input



Convolutional layer

- Once we have seen
  - Convolutional kernels can be learned in order to extract features that may be useful the task of classification (or regression)
  - Convolutions can be equipped with padding/stride
- Convolutional layer: the most important layer in a CNNs
- When we define a convolutional layer we must define
  - Number of input channels
  - Number of output channels (number of convolutions)
  - Kernel size (only height and width)
  - Stride (by default 1)
  - Padding (by default 0)

Convolutional layer

- The convolutional layer will learn
  - Kernel values (weights)
  - Bias: as many as output channels
- Usually, a ReLU activation function must be added after the convolutional layer
- In PyTorch
  - nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1, padding=1)

Convolutional layer

- Thinking on tensors
  - Input: C_in x H x W
  - The kernel of a conv. Layer: C_out x C_in x k x k
  - Bias: C_out
  - Output: C_out x H' x W'
    - Considering padding/stride
- In practice, we process mini-batches of N images, so
  - Input: N x C_in x H x W
  - Output: N x C_out x H' x W'

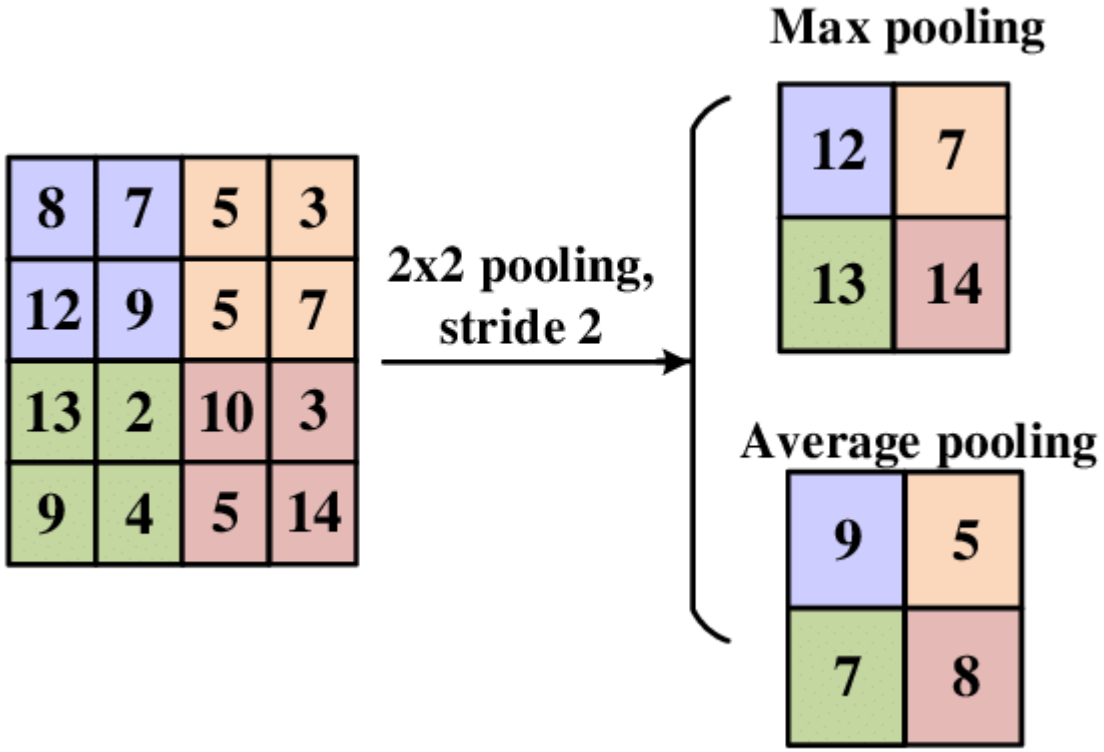
Pooling layers

- Images have usually more than 8x8 pixels
  - For example, a grayscale 28x28 pixels has 784 values
  - An RGB 128x128 image has 49,152 values
- Convolutional layers do not reduce this amount; they usually increase it
  - Applying 16 convolutions with padding 1, we will go from 28x28 to 16x28x28 = 12,544 alues
- Suppose we now flatten and feed into an MLP with 128 neurons
  - Number of paremeters: 12,544 x 128 + 128 > 1,6M
- However, neighbouring pixels are usually very similar (redundant information=
- Idea: summarize each small region with a single feature -> pooling

Pooling layer

- Pooling layers are the second type of usual layers in CNNs.
- As convolutional layers, pooling operators consist of a fixed-shape window that is slide over all regions, computing a single output for each location
- No kernel is used, since the operation performed is deterministic
  - Maximum in the case of a max-pooling
  - Average in the case of a avg-pooling
- Pooling layers are used to downsample the image, while trying to preserve the extracted features from the convolutional layers
- Usual definition
  - 2x2 window
  - Stride of 2

Pooling layer



Pooling layer

- In PyTorch
  - pool = nn.MaxPool2d(kernel_size=2, stride=2)
  - pool = nn.AvgPool2d(kernel_size=2, stride=2)
- Input: N x C x H x W; Ouput: N x C x H/2 x W/2